“Linux System Debugging Tool”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

DHARMIKA	**2520030012**
ANUSHKA K	**2520030501**
RUSHVINI	**2520030542**

Under the Esteemed Guidance of

Harika
Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux System Debugging Tool"** is a record of Bonafide work of **Dharmika - 2520030012, Anushka K - 2520030501, Rushvini - 2520030542** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering and Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

DHARMIKA	**2520030012**
ANUSHKA K	**2520030501**
RUSHVINI	**2520030542**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Linux System Debugging Tool"** is a bonafide work done and submitted by **Dharmika - 2520030012, Anushka K - 2520030501, Rushvini - 2520030542** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator Dr. K. Anusha and our guide Harika, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science and Engineering***, for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

DHARMIKA **2520030012**

ANUSHKA K **2520030501**

RUSHVINI **2520030542**

Abstract

Diagnosing why a program misbehaves on Linux is difficult for students because the useful information (which system calls a program makes, what its CPU registers hold, what its memory contains, how many resources it consumes) lives inside the kernel and is not visible in the normal output of the program. Professional tools such as `strace` and `gdb` expose this information, but they are large and hide the kernel mechanisms they rely on.

This project presents the **Linux System Debugging Tool**, a compact debugger written in C that obtains all its information directly from kernel interfaces. The debugger starts a target program as a child process with `fork()`, places it under kernel tracing with `ptrace(PTRACE_TRACEME)`, synchronises with it using `waitpid()` and the `SIGSTOP` signal, and then resumes it with `PTRACE_SYSCALL` so that it stops at every system call entry and exit. At each entry the tool reads the CPU registers with `PTRACE_GETREGSET`, reads 64 bytes of the target's stack with `process_vm_readv()`, and finally reports CPU time and memory figures with `getrusage()` and `sysinfo()`.

A small web server written in C (sockets, `popen()`) runs the debugger on request, saves every session as a time-stamped log, lists the saved sessions, lists the threads of any running process by reading `/proc/PID/task`, and serves a browser dashboard built with HTML, CSS and JavaScript. Three programs can be debugged from the dashboard: the sample `target_program`, `ls` and `whoami`; a three-thread POSIX program (`thread_demo`) demonstrates the thread monitor.

In the recorded execution on the default target the tool detected **33 system call entries**, read the registers and memory, and reported a CPU user time of 0.006705 seconds, a maximum memory of 1408 KB and 7724 MB of total RAM. The history page listed **64 saved debugging sessions**. The project demonstrates user/kernel interaction, process creation and control, signals, tracing, direct address-space reads, threads, inter-process communication and a client-server design in one small, readable code base.

Keywords: Linux, debugging, ptrace, system calls, process control, signals, process_vm_readv, POSIX threads, /proc, getrusage, sysinfo, web dashboard.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		**i**
	Abstract		**ii**
	Index		**iii**
1	**Introduction**		**1**
		1.1 Background of the Project	1
		1.2 Problem Statement	2
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	3
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	4
2	**OS Concepts and Background Study**		**5**
		2.1 Operating System Concepts Related to the Project	5
		2.2 Existing Methods / Algorithms	7
		2.3 Proposed Approach	7
		2.4 Advantages of the Proposed Approach	8
3	**System Requirements**		**9**
		3.1 Hardware Requirements	9
		3.2 Software Requirements	9
		3.3 Operating System Environment - Ubuntu / WSL	9
		3.4 Development Tools and Compilers	10
4	**Technology and Tools Used**		**11**
		4.1 C / C++ Programming	11
		4.2 GCC Compiler	11
		4.3 Linux System Calls	12
		4.4 POSIX Threads / Synchronization	13
		4.5 Linux Commands and Utilities	13
		4.6 Other Tools / Libraries Used	14
5	**System Architecture and Design**		**15**
		5.1 Overall System Architecture	15
		5.2 System Workflow	16
		5.3 Flowchart / Data Flow	16
		5.4 Module Description	19
		5.5 Input and Output Design	19
6	**Implementation**		**21**
		6.1 Module-wise Implementation	21
		6.2 Algorithm / Pseudocode	22
		6.3 Core C / C++ Implementation	23

S. No.	Chapter	Topics / Subtopics	Page No.
		6.4 Linux System Calls / APIs Used	33
		6.5 Process / Thread Implementation	34
		6.6 IPC / Synchronization Implementation	34
		6.7 Memory Management Implementation	35
		6.8 Optional User Interface Implementation	36
7	**Execution and Demonstration**		**37**
		7.1 Compilation in Ubuntu	37
		7.2 Execution Commands	37
		7.3 Sample Inputs and Outputs	38
		7.4 Screenshots of Execution	38
		7.5 Demonstration of OS Concepts	42
8	**Conclusion**		**43**
		8.1 Summary of the Project	43
		8.2 Objectives Achieved	43
		8.3 Results Achieved	43
		8.4 Skills and Learning Outcomes	44
		8.5 Limitations and Future Scope	44
9	**References**		**46**
		9.1 Books	46
		9.2 Research Papers	46
		9.3 Linux / GNU Documentation	46
		9.4 Online Resources	46

CHAPTER - 1
INTRODUCTION

1.1 Background of the Project

Linux is the most widely deployed operating system kernel in the world. It runs the servers behind websites and cloud platforms, the supercomputers used for scientific research, the Android phones in our pockets and the embedded controllers inside cars and televisions. Because so many critical systems depend on Linux, the ability to understand what a Linux program is doing, and to find out why a program misbehaves, is one of the most valuable skills a computer engineer can have.

When a program on Linux fails, the visible symptom is usually small: a window that does not open, a command that prints “Segmentation fault”, a server that is slow, or a file that cannot be read. The real cause, however, lies deeper, inside the interaction between the program and the operating system: a pointer that refers to memory that was never mapped, a system call that returns an error nobody checked, or a program that keeps asking the kernel for resources it does not need. Finding such causes is called **debugging**, and doing it well requires visibility into the operating system itself.

Linux provides this visibility through a small number of well-defined kernel interfaces. The `ptrace` system call lets one process observe and control another: it can stop the other process at every system call, read its CPU registers and examine its memory. **Signals** tell a process about events such as a stop request or a trap. The `process_vm_readv` system call copies data directly from the address space of another process. `getrusage` and `sysinfo` report how much processor time and memory a process and the whole machine are using. The `/proc` pseudo file system exposes the process table, including the list of threads of every process. Professional tools such as `strace`, `gdb`, `ltrace`, `top` and `valgrind` are all built on these same interfaces.

In an Operating Systems course these ideas are studied mostly in theory: students learn that a process has a state, that system calls cross the user/kernel boundary and that signals are delivered asynchronously, but they rarely see these mechanisms work together inside one program. This project, the **Linux System Debugging Tool**, closes that gap. It is a small debugger, written in C, that starts a program under kernel tracing, records every system call, shows the CPU registers and a piece of stack memory, reports resource usage, and presents all of this in a browser dashboard together with a thread monitor and a history of earlier debugging sessions.

The tool was developed and tested on Linux using the GNU C compiler. The debugger itself is a plain command-line program of about 400 lines in six C files; the web server is a single C file and the dashboard is three small web files. No external library is needed apart from the standard C library and POSIX threads, which keeps every line of the project open to inspection and explanation.

1.1.1 What is debugging?

Debugging is the systematic process of locating and removing the cause of incorrect behaviour in software. For operating-system level problems four complementary activities are normally used, and the tool supports each of them:

- **Observation:** watching the system calls that a program makes, which shows what the program asks the kernel to do (open files, allocate memory, write output).
- **Inspection:** reading the CPU registers and the memory of the program at a chosen moment, which shows the state of the computation.
- **Measurement:** collecting CPU time and memory figures, which show how expensive the program is.
- **Review:** keeping a log of every session so that results can be compared later.

1.2 Problem Statement

Understanding why a program fails or how it uses the operating system cannot be done from its ordinary output. Runtime failures such as segmentation faults, unhandled exceptions or unexpected delays need information about kernel interactions, CPU register contents and the memory of the process. Heavyweight debuggers can provide it, but they are difficult for a beginner: their command sets are large, and they hide the very mechanisms (ptrace, signals, `waitpid`) that an Operating Systems student needs to learn.

The problem addressed by this project is therefore to **design and build a lightweight Linux debugging tool** that (a) launches a target program under kernel control, (b) intercepts and records its system calls, (c) reads CPU registers and process memory, (d) measures resource usage, (e) can list the threads of a running process, and (f) presents the result in a form that is easy to read and keeps a permanent history. The tool must be small enough that the complete source can be understood by a student, and safe enough that it can only debug programs chosen from a fixed list.

1.3 Scope of the Project

The scope defines what the tool does and, equally important, what it intentionally does not do.

1.3.1 Within the scope

- Creation of a traced child process with `fork()`, `PTRACE_TRACEME`, `raise(SIGSTOP)` and `execl()`.
- System call tracing with `PTRACE_SYSCALL`: every entry and exit stop of the target is observed and every entry is reported with its system call number.
- Reading of the CPU registers (X0 to X8, SP, PC, PSTATE) with `PTRACE_GETREGSET`.
- Reading 64 bytes from the top of the target's stack with `process_vm_readv()` and printing them in hexadecimal.
- Resource reporting with `getrusage()` (CPU user time, CPU system time, maximum memory) and `sysinfo()` (total, free and available RAM).
- A C web server on port 8080 that runs the debugger, stores each session as a log file, lists the history, shows a log, reports the server's PID and PPID and lists the thread IDs of a process from `/proc/PID/task`.
- A browser dashboard (HTML, CSS and JavaScript) with status cards, a debugger summary, a process information panel, a POSIX thread monitor and a debugging history table.

- Three selectable targets: `target_program`, `ls` and `whoami`; and a three-thread demonstration program, `thread_demo`.

1.3.2 Outside the scope

- Breakpoints, single-stepping and symbolic debugging with source-line information, as provided by `gdb`.
- Decoding of system call arguments and return values; only the system call number is reported.
- Debugging of arbitrary user-chosen programs; for safety only the three allow-listed targets can be started from the dashboard.
- Remote or multi-user use; the server is intended for `localhost` and has no authentication.
- Support for processor families other than the ARM64 (aarch64) register layout used by the supplied build.

1.4 Objectives of the Project

1. To study and apply the Linux kernel interfaces for tracing and controlling a process: `ptrace`, `fork`, `execl`, `waitpid` and signals.
2. To intercept every system call of a child process with `PTRACE_SYSCALL` and to separate entry stops from exit stops.
3. To read the CPU register state of a traced process with `PTRACE_GETREGSET` and interpret the system call number held in register X8.
4. To read the live memory of another process directly with `process_vm_readv()` without using `PTRACE_PEEKDATA`.
5. To measure CPU time and memory use with `getrusage()` and `sysinfo()`.
6. To design a modular code base in which process control, tracing, register reading, memory reading and resource monitoring are separate source files.
7. To build a client-server layer in C (sockets, `popen`) so that the debugger can be started from a web browser, with an allow-list that prevents arbitrary programs from being launched.
8. To list the threads of a running process by reading `/proc/PID/task` and to demonstrate this with a three-thread POSIX program.
9. To store each debugging session as a time-stamped log and to show the history of sessions in the dashboard.
10. To document the design, implementation and the observed results of the project.

1.5 Importance of the Application

The importance of the application lies in four areas.

- **Educational value.** Each concept of the Operating Systems syllabus (user and kernel space, the process life cycle, signals, system calls, virtual memory, threads and inter-process communication) appears in a short, readable piece of real code that can be run and observed.

- **Practical diagnostic value.** The system call list and resource figures answer basic questions such as “what does this program ask the kernel to do?” and “how much memory does it need?” without installing any additional software.
- **Transparency.** Because the tool calls the kernel interfaces directly, students see exactly how a tool like `strace` works inside.
- **Usability.** The browser dashboard turns a command-line trace into status cards and a readable summary, and the history table keeps earlier results available for comparison.

1.6 Target Users / Audience

User group	How they use the tool
Operating Systems students	Run the debugger on the sample program, observe the system call numbers, registers and stack bytes, and relate them to the theory taught in class.
Laboratory instructors	Use the dashboard as a live demonstration of ptrace, signals, process creation and thread listing during lectures and practical sessions.
Beginner Linux developers	Check quickly which system calls a small program makes and how much CPU time and memory it uses.
Project evaluators and reviewers	Use the history page and saved logs to review the sessions that were run during development.

1.6.1 Organisation of the report

Chapter 1 introduced the project. Chapter 2 reviews the Operating System concepts and existing techniques on which the tool is based. Chapter 3 lists the hardware and software requirements, and Chapter 4 describes the technology and tools. Chapter 5 explains the architecture and design with diagrams. Chapter 6 describes the implementation module by module with the source code. Chapter 7 shows compilation, execution and the screenshots of the real output. Chapter 8 concludes with the results, learning outcomes, limitations and future scope, and Chapter 9 lists the references.

CHAPTER - 2
OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

This section explains the Operating System concepts that the tool uses. Each concept is tied to the part of the project in which it appears.

2.1.1 User space and kernel space

The processor runs code in at least two privilege levels. Ordinary programs run in **user mode**, where they cannot access hardware or the memory of other processes. The operating system kernel runs in **kernel mode** with full privileges. A user program asks the kernel for a service (reading a file, allocating memory, creating a process) through a **system call**, which switches the processor into kernel mode, runs the kernel code and returns. A debugger is interesting because it is a user program that, with the help of the kernel, can observe these crossings of the boundary in another program.

2.1.2 Processes and their life cycle

A **process** is a program in execution: it has its own virtual address space, its own set of open files and an entry in the kernel's process table identified by a process ID (PID). Every process except the first has a parent process (PPID). A new process is created by `fork()`, which makes a copy of the caller; the copy can then replace its program image with `execl()` or a related call. The parent can wait for the child with `waitpid()`, which also reports whether the child stopped, exited normally or was killed by a signal. The debugger uses exactly this life cycle: it forks the target, lets the child stop, and then supervises it until it exits. The dashboard's process information panel shows the PID and PPID of the server process.

2.1.3 System calls

A system call is identified by a number. On the ARM64 architecture the number is placed in register X8 and the arguments in X0 to X5 before the processor executes the `svc` instruction. Because the number is in a known register, a tracer can read it at the moment the call is made. For example, number 221 is `execve`, 214 is `brk`, 222 is `mmap`, 56 is `openat`, 80 is `fstat` and 57 is `close`. Table 4.1 in Chapter 4 lists the numbers that appear in the project's recorded output.

2.1.4 The ptrace mechanism

`ptrace` (process trace) is the system call on which debuggers are built. A process becomes a **tracee** either by calling `ptrace(PTRACE_TRACEME)` on itself, or by being attached to by another process. The process that controls it is the **tracer**. Once a process is traced, the kernel stops it whenever a signal is delivered to it and the tracer is informed through `waitpid()`. The tracer can then read and modify registers and memory and decide how the tracee continues. The request `PTRACE_SYSCALL` resumes the tracee and makes the kernel stop it again at the next system call entry or exit. This is the request on which the system call tracer of the project is based.

2.1.5 Signals

A **signal** is a short notification delivered by the kernel to a process. `SIGSTOP` asks a process to stop; `SIGTRAP` is used by the kernel to report debugging events such as the completion of an `exec`; `SIGSEGV` reports an invalid memory access. In the project the child raises `SIGSTOP` on itself, immediately after enabling tracing, so that the parent can attach its tracing logic to a process that is known to be stopped and has not yet executed the target program. After the stop, `WIFSTOPPED(status)` and `WSTOPSIG(status)` from `waitpid` confirm it, and the debugger prints the signal number.

2.1.6 Virtual memory and cross-process reads

Every process sees a private virtual address space that the memory manager maps to physical memory. The same address value in two processes therefore refers to different data. To inspect a target the tracer needs the target's addresses interpreted in the target's address space. `process_vm_readv()` does this: the caller gives a local buffer and a remote address, and the kernel copies the data directly between the two address spaces in a single system call. The project reads the target's stack by taking the stack pointer (SP) from the registers and requesting 64 bytes at that address.

2.1.7 Threads

A **thread** is a unit of execution inside a process. All threads of a process share the address space and the open files but each has its own stack, registers and thread ID (TID). On Linux a POSIX thread is created with `pthread_create()`; the kernel represents each thread as a task, so the threads of a process appear as numbered directories under `/proc/PID/task`. The first thread has a TID equal to the PID. The thread monitor of the dashboard reads this directory, and the supplied `thread_demo` program creates three worker threads to give it something to list.

2.1.8 The /proc file system

`/proc` is a pseudo file system: its files are produced by the kernel on demand and describe the current state of the system. A directory `/proc/PID` exists for every process, and `/proc/PID/comm` holds the command name. The server reads `/proc/PID/comm` to find a running `thread_demo` process and `/proc/PID/task` to list threads.

2.1.9 Resource accounting

The kernel keeps account of the processor time and memory used by every process. `getrusage()` returns these figures: CPU time spent in user mode (`ru_utime`), CPU time spent in kernel mode (`ru_stime`) and the maximum resident set size (`ru_maxrss`, in kilobytes on Linux). `sysinfo()` returns system-wide figures such as total and free RAM. They are the basis of the "System Resource Details" panel.

2.1.10 Inter-process communication and sockets

Processes cooperate through inter-process communication (IPC). The project uses three forms: an **environment variable** (`TARGET_PROGRAM`) inherited by the debugger from the server; a **pipe** created by `popen()` through which the debugger's output reaches the server; and a **TCP socket** on which the server talks to the browser using the HTTP protocol. Section 6.6 describes them in detail.

Concept	Where it appears in the project
User space / kernel space	Every ptrace, waitpid and process_vm_readv call crosses the boundary
Process creation and life cycle	fork, execl, waitpid in process_control.c; popen in server.c
System calls	PTRACE_SYSCALL tracing in syscall_tracer.c
Signals	SIGSTOP raised by the child; WIFSTOPPED / WSTOPSIG in the parent
Virtual memory	process_vm_readv of the target stack in memory_reader.c
Threads	thread_demo.c and the thread monitor (/proc/PID/task)
Resource accounting	getrusage and sysinfo in resource_monitor.c
IPC and sockets	Environment variable, popen pipe, TCP socket on port 8080

2.2 Existing Methods / Algorithms

Several established tools already show what a process does. They differ in the interface they use and in their complexity.

Tool	What it does	Kernel interface	Limitation for a learner
strace	Prints every system call with arguments and return value	ptrace (PTRACE_SYSCALL / seccomp)	Large code base; the mechanism is hidden inside the tool
gdb	Breakpoints, stepping, source-level debugging	ptrace, debug information	Large command language; steep learning curve
ltrace	Traces calls to shared-library functions	ptrace, dynamic linker data	Does not show kernel interaction directly
top / ps	Show processes and their resource use	/proc	No tracing and no register or memory view
valgrind	Detects memory errors by running the program on a virtual CPU	Dynamic binary instrumentation	Slow; complex internals
perf	Performance counters and profiling	perf_event_open	Requires privileges; statistical rather than exact

All of these tools use one or more of the same few kernel interfaces. The algorithm shared by tracers is the **trace loop**: start the tracee, resume it until its next event, wait for the event, examine the state, and repeat until the tracee exits. This is the algorithm implemented in `syscall_tracer.c`.

2.3 Proposed Approach

The proposed approach is to build a small tracer that uses the kernel interfaces directly and to put a simple web interface in front of it. The main design decisions are:

1. **The kernel is the single source of truth.** Registers come from `PTRACE_GETREGSET`, memory from `process_vm_readv`, resources from `getrusage` and `sysinfo`, and threads from `/proc`. Nothing is guessed or simulated.
2. **Modular structure.** The debugger is split into process control, the system call tracer, the register reader, the memory reader and the resource monitor, each in its own file, and `debugger.c` only calls them in order.
3. **A restricted set of targets.** The debugger and the server both check the requested program against an allow-list (`./target_program`, `/bin/ls`, `/usr/bin/whoami`), so the web interface

cannot be used to start arbitrary programs.

4. **A separate server process.** The server runs the debugger as a child with `popen()`, collects its text output, stores it and returns it, so a crash or delay in the debugger cannot take the server down.
5. **Plain, checkable output.** The debugger prints a text report with fixed labels; the dashboard extracts the values by matching these labels, and the same text is saved as a log.

2.4 Advantages of the Proposed Approach

- **Small and readable:** the debugger is about 400 lines in six files.
- **No dependencies:** only the standard C library, POSIX interfaces and a web browser are needed.
- **Direct kernel access:** the tool shows how real tracers work instead of hiding the mechanisms.
- **Safe by design:** an allow-list limits the programs that can be started, and the server validates process IDs and log file names before using them.
- **Reviewable:** every session is saved as a log, and the history page lists them with the target program.
- **Easy to extend:** a new module (for example system call names or breakpoints) can be added as one more file that `debugger.c` calls.

CHAPTER - 3
SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The tool is small and has modest hardware needs. The figures in the table are the values recorded in the execution of Chapter 7 and the values recommended for running the project.

Component	Recorded in the execution	Recommended
Processor	ARM64 (aarch64) Linux system; the supplied binaries are ARM64 executables	Any 64-bit ARM (aarch64) processor, because the register code uses the ARM64 register layout
Main memory (RAM)	7724 MB total, 6315 MB free (from the dashboard)	2 GB or more
Disk space	Source, binaries and 64 saved logs are a few hundred kilobytes in total	50 MB free for the project and logs
Display	Any monitor able to show a web browser	1280 x 720 or larger for the dashboard
Network	Loopback only (localhost:8080)	No external network is required

3.2 Software Requirements

Software	Purpose
Linux (Ubuntu or a similar distribution)	Provides ptrace, process_vm_readv, /proc and POSIX threads
GNU C compiler (gcc) and the C library headers	Compiles the debugger, server and the two sample programs
POSIX threads library (libpthread, part of glibc)	Needed by thread_demo (compile with -pthread)
Linux kernel headers (asm/ptrace.h, elf.h)	Provide struct user_pt_regs and NT_PRSTATUS used by the register and memory modules
A modern web browser	Runs the dashboard (HTML, CSS and JavaScript with the fetch API)
Terminal / shell	Starts the server and, optionally, the debugger directly
Git (optional)	Version control and the project repository on GitHub

3.3 Operating System Environment - Ubuntu / WSL

The project needs a real Linux kernel because it depends on kernel interfaces that do not exist on other systems. The supplied executables are dynamically linked ELF files for 64-bit ARM, built for GNU/Linux 3.7.0 or later, and use the loader `/lib/ld-linux-aarch64.so.1`.

3.3.1 Native Ubuntu or another Linux distribution

On a native Linux installation the project runs without any further preparation. The debugger traces a child that it has created itself, which is allowed under the default Yama `ptrace_scope` setting of Ubuntu, because a parent may trace its own children.

3.3.2 Windows Subsystem for Linux (WSL)

Students who use Windows can run the project under WSL 2, which provides a real Linux kernel. Install a distribution from the Microsoft Store or with `wsl --install -d Ubuntu`, open the Ubuntu terminal and install the compiler with `sudo apt install build-essential`. The dashboard can then be opened from the Windows browser, because WSL forwards `localhost` to the Linux side.

3.3.3 Useful commands for checking the environment

Checking the Linux environment

```
uname -m                # architecture; the supplied build needs aarch64
uname -r                # kernel release
gcc --version           # compiler is installed
cat /proc/sys/kernel/yama/ptrace_scope    # 1 = a parent may trace its children
free -m                 # memory figures that sysinfo() also reports
```

3.4 Development Tools and Compilers

Option / tool	Meaning	Why it is used
gcc -Wall	Enables the common compiler warnings	Catches unused variables and type mistakes early
gcc -g	Adds debugging information	Allows the project to be examined with gdb while it is developed
gcc -o name	Names the output executable	Produces debugger, server, target_program and thread_demo
gcc -pthread	Links POSIX threads	Required by thread_demo.c
mkdir -p logs	Creates the log directory	The server writes every session to logs/debug_*.log

CHAPTER - 4
TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The entire back end of the project is written in **C** (the C11 dialect used by GCC). C was chosen deliberately: it is the language of the Linux kernel and of its system call interface, every system call used in the project is declared in a C header, and structures such as `struct user_pt_regs`, `struct iovec`, `struct rusage` and `struct sysinfo` are defined for C programs.

- **Direct access to the kernel interface.** System calls such as `ptrace`, `waitpid` and `process_vm_readv` are called directly, without any wrapper library.
- **Predictable resource use.** C has no garbage collector or runtime, so memory use and timing are determined by the program text, which matters for a tool that measures resources.
- **Small dependency footprint.** The tool needs only the standard C library, so the whole code base can be read and understood.
- **Educational transparency.** Every system call and every buffer is visible in the source.

C++ was considered because it offers classes and RAII, but it was not needed: the design is a set of small functions called in sequence, and the project is meant to show the plain C interface of the kernel. The front end uses **HTML, CSS and JavaScript**, described in Section 4.6.

4.1.1 C language features used

Feature	Where it is used
Structures (`struct`)	user_pt_regs for registers; iovec for process_vm_readv and PTRACE_GETREGSET; rusage and sysinfo for resources; sockaddr_in in the server
Pointers and pointer arithmetic	Parsing request lines in the server (`start[i]`, `strchr`, `strstr`) and passing buffer addresses to the kernel
Fixed-size arrays and `sizeof`	Buffers such as `unsigned char buffer[64]` and `char result[BUFFER_SIZE]`; `sizeof` is used for lengths
Function declarations across files	`read_registers` and `read_process_memory` are declared in syscall_tracer.c and defined in their own files
Preprocessor	`#define PORT 8080`, `#define DEFAULT_PROGRAM`, `#define _GNU_SOURCE` (needed for process_vm_readv)
Bounded string functions	`snprintf`, `strncmp` and length checks before `memcpy` and `strcat`
Status macros	`WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`

4.2 GCC Compiler

The **GNU Compiler Collection (GCC)** translates the C source into an executable ELF file in four stages: pre-processing (headers and macros), compilation (to assembly), assembly (to an object file) and linking (with the C library and, for thread_demo, the pthread library). The project is built from separate source files, so the debugger is produced by compiling the six files in `src/debugger` together and linking them into one executable named `debugger`.

Stage	Command	Result
Pre-processing	gcc -E file.c	Headers inserted and macros expanded
Compilation	gcc -S file.c	Assembly code (ARM64 for the supplied build)
Assembly	gcc -c file.c	Relocatable object file file.o
Linking	gcc *.o -o debugger	Executable ELF file linked with the C library

4.3 Linux System Calls

System calls are the interface between the project and the kernel. Table 4.1 lists the calls and library functions used and their role.

System call / function	Header	Used for	Source file
fork()	unistd.h	Create the child that will run the target	process_control.c
ptrace(PTRACE_TRACEME)	sys/ptrace.h	Child asks to be traced by its parent	process_control.c
raise(SIGSTOP)	signal.h	Child stops itself so the parent can take control	process_control.c
execl()	unistd.h	Replace the child with the target program	process_control.c
waitpid()	sys/wait.h	Wait for the child to stop, exit or be killed	process_control.c, syscall_tracer.c
ptrace(PTRACE_SYSCALL)	sys/ptrace.h	Resume the child until the next system call entry/exit	syscall_tracer.c
ptrace(PTRACE_GETREGSET)	sys/ptrace.h	Read the CPU registers (NT_PRSTATUS)	syscall_tracer.c, register_reader.c, memory_reader.c
process_vm_readv()	sys/uio.h	Copy 64 bytes of the child’s stack	memory_reader.c
getrusage()	sys/resource.h	CPU user/system time and maximum memory	resource_monitor.c
sysinfo()	sys/sysinfo.h	Total and free RAM	resource_monitor.c
socket, bind, listen, accept	sys/socket.h	HTTP server on port 8080	server.c
popen() / pclose()	stdio.h	Run ./debugger and read its output through a pipe	server.c
opendir(), readdir()	dirent.h	List /proc, /proc/PID/task and the logs directory	server.c
setsid(), dup2(), open()	unistd.h, fcntl.h	Start thread_demo detached from the server	server.c
pthread_create(), pthread_join()	pthread.h	Create and wait for three threads	thread_demo.c
getpid(), getppid()	unistd.h	Report PID and PPID	thread_demo.c, server.c

Table 4.1: System calls and functions used in the project.

4.3.1 System call numbers seen in the recorded output

The dashboard lists the first ten system call numbers that the tracer reported for `target_program`: 221, 0, 214, 222, 48, 56, 80, 222, 57 and 56. Table 4.2 gives the meaning of the numbers on ARM64 according to the generic Linux system call table.

Number	System call	Meaning
221	execve	Load the target program into the child process

Number	System call	Meaning
214	brk	Set up or grow the heap
222	mmap	Map memory, for example the program and shared libraries
48	faccessat	Check whether a file can be accessed (for example library configuration files)
56	openat	Open a file
80	fstat	Obtain file information
57	close	Close a file descriptor
0	io_setup	Not expected from a normal program; see Section 8.5 for the likely reason it appears

Table 4.2: ARM64 system call numbers found in the recorded output.

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) give a C program several flows of control inside one process. The project uses them in `thread_demo.c`, a demonstration program that creates three worker threads with `pthread_create()`. Each worker receives a numeric ID through its argument, prints a start message, performs three steps with a 15-second `sleep()` between them, and prints a finish message. The main thread then waits for each worker with `pthread_join()`, which is the synchronisation primitive that guarantees the process does not exit before its threads are finished. The long sleeps keep the process alive for about 45 seconds, long enough to look up its PID in the dashboard and list its threads.

The debugger and the server themselves are single-threaded: the server handles one connection at a time in a loop of `accept()`, and the debugger traces one child at a time. No shared data is therefore modified by two threads at once, and no mutex is needed. Chapter 6 explains this in Section 6.6.

4.5 Linux Commands and Utilities

Command	Purpose in the project
gcc	Compile the C sources
mkdir -p logs	Create the directory in which the server stores logs
./server	Start the web server on port 8080
./debugger	Run the debugger directly in the terminal
./thread_demo &	Start the three-thread demonstration in the background
ps, pidof, pgrep	Find the PID of a running program to enter in the thread monitor
ls /proc/PID/task	Check the thread IDs shown by the dashboard
uname, free, cat /proc/meminfo	Cross-check system and memory figures
strace (optional)	Compare the reported system call numbers with a professional tool
curl (optional)	Test server endpoints such as /process-info without a browser

4.6 Other Tools / Libraries Used

Tool / library	Role
HTML5	Structure of the dashboard page (index.html): header, control panel, status cards, resource panel, summary, process information, thread monitor and history table
CSS3	Styling of the dashboard (style.css): cards, grid layout, dark summary panel, buttons and the history table
JavaScript (fetch API)	script.js sends requests to the C server, extracts values from the debugger text with regular expressions and updates the page
HTTP / TCP sockets	Communication between the browser and the C server on localhost:8080, with CORS headers so that the page can be opened from a file
Git and GitHub	Source control; repository: github.com/Dharmika-12/KLH-CSE-2026-27-Team15-LinuxSystemDebuggingTool
Web browser developer tools	Used to check requests and responses during development

CHAPTER - 5
SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system is organised in six layers. The upper layers are written for the project; the lower layers are the Linux interfaces on which it stands. A request travels downwards from the browser to the kernel and the result travels back up as text.

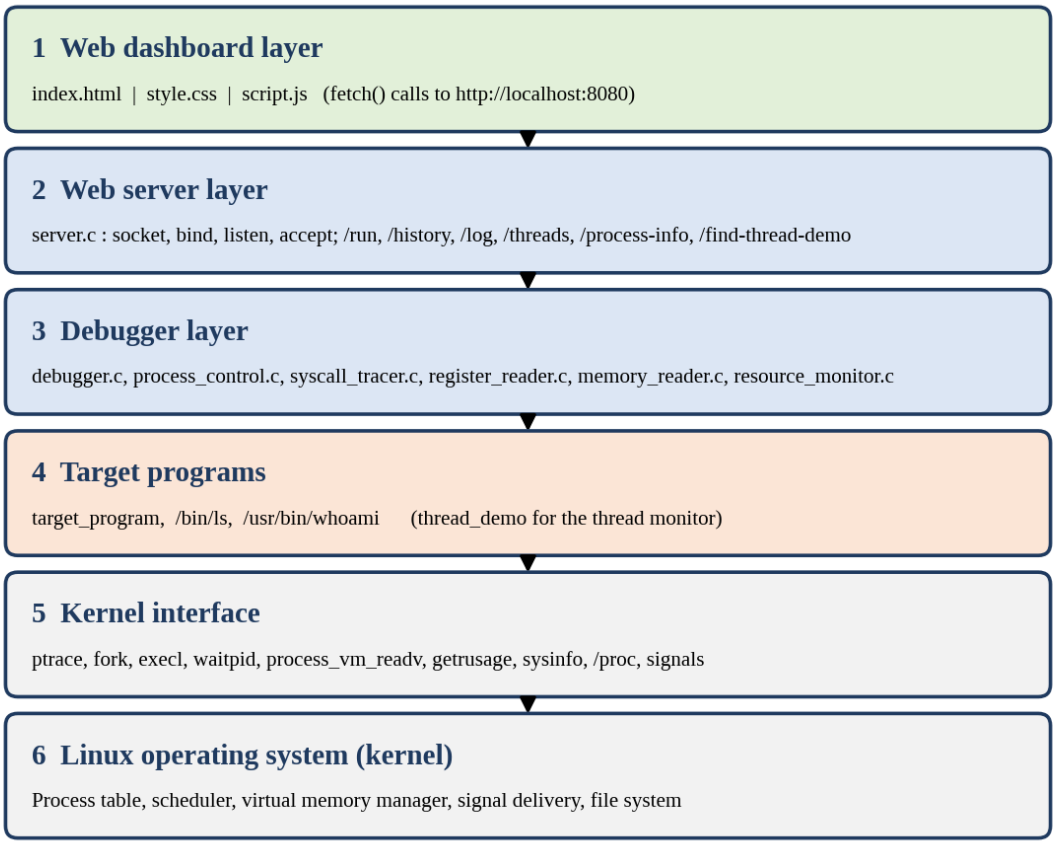


Figure 5.1: Layered architecture of the Linux System Debugging Tool

Layer	Components	Responsibility
1 Web dashboard	index.html, style.css, script.js	Select a target, show status cards, the summary, process information, thread monitor and history
2 Web server	server.c	Accept HTTP requests, run the debugger, save and list logs, report PIDs, list threads
3 Debugger	debugger.c, process_control.c, syscall_tracer.c, register_reader.c, memory_reader.c, resource_monitor.c	Start, trace and inspect the target and print a text report
4 Targets	target_program, ls, whoami, thread_demo	The programs that are debugged or observed
5 Kernel interface	ptrace, fork, execl, waitpid, process_vm_readv, getrusage, sysinfo, /proc, signals	The only source of the information the tool shows
6 Operating system	Process table, scheduler, virtual memory manager, signal delivery	Provides and enforces everything above

5.2 System Workflow

One debugging session passes through the following steps (Figure 5.2).

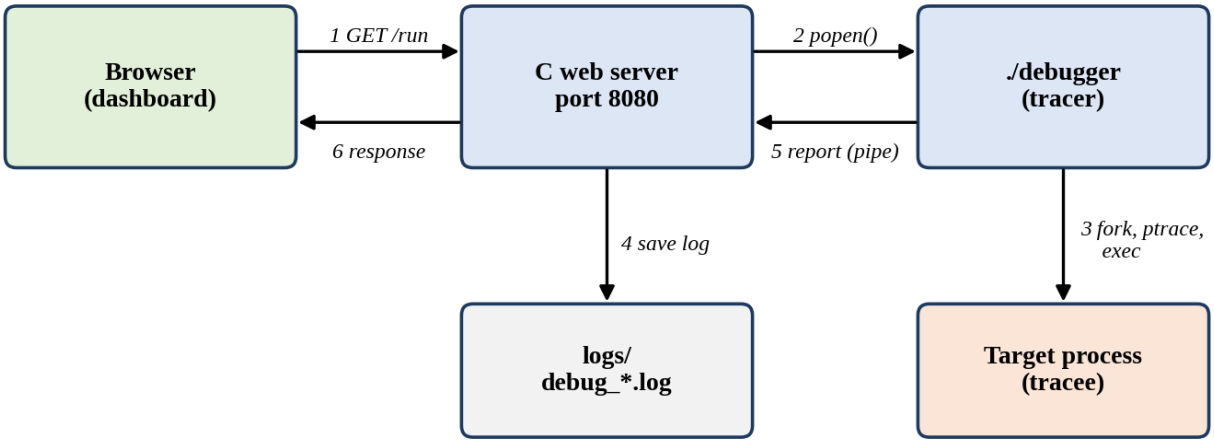


Figure 5.2: Workflow of one debugging session from the browser to the target process

1. The user selects a target in the dashboard and presses **Start Debugger**. The page sets the cards to “Running” and sends `GET /run?program=NAME` to the server.
2. The server checks that the name is `target_program`, `ls` or `whoami`, maps it to `./target_program`, `/bin/ls` or `/usr/bin/whoami`, sets the environment variable `TARGET_PROGRAM` and starts `./debugger` with `popen()`.
3. The debugger reads `TARGET_PROGRAM`, checks it against its own allow-list, forks the child and traces it as shown in Figure 5.4.
4. During tracing the debugger prints a line for every system call entry together with the registers and the stack bytes. At the end it prints the resource monitor section and “Debugging completed”.
5. The server reads the whole output through the pipe, writes it to `logs/debug_YYYYMMDD_HHMMSS.log` with the first line “Selected program: NAME”, and returns it to the browser.
6. The page extracts the target name, system call count, module status and resource figures from the text and updates the cards, the resource panel and the debugger summary.

5.3 Flowchart / Data Flow

5.3.1 System call tracing loop

The core of the debugger is the loop in `trace_system_calls()`. Figure 5.3 shows its logic. The flag `entering` starts as 1 and is inverted after every stop, so that the loop can tell entry stops from exit stops.

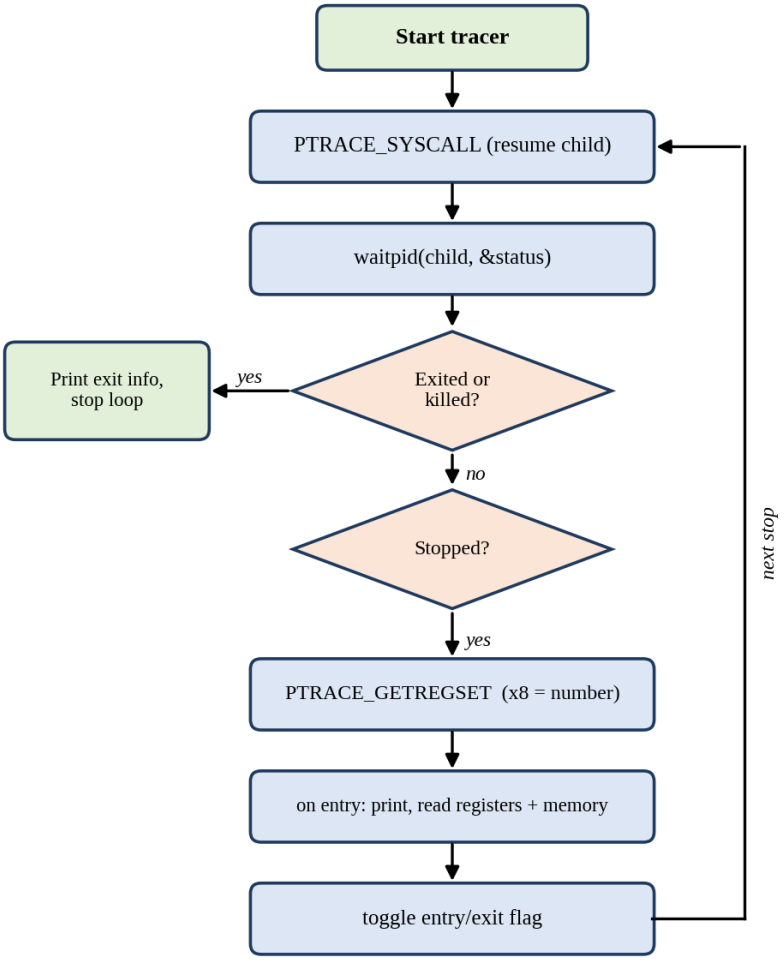


Figure 5.3: Flowchart of the system call tracing loop

5.3.2 Parent and child

For tracing the tool runs as two processes: the debugger (the tracer) and the child (the tracee). Figure 5.4 shows the order of the operations that bring the child under control.

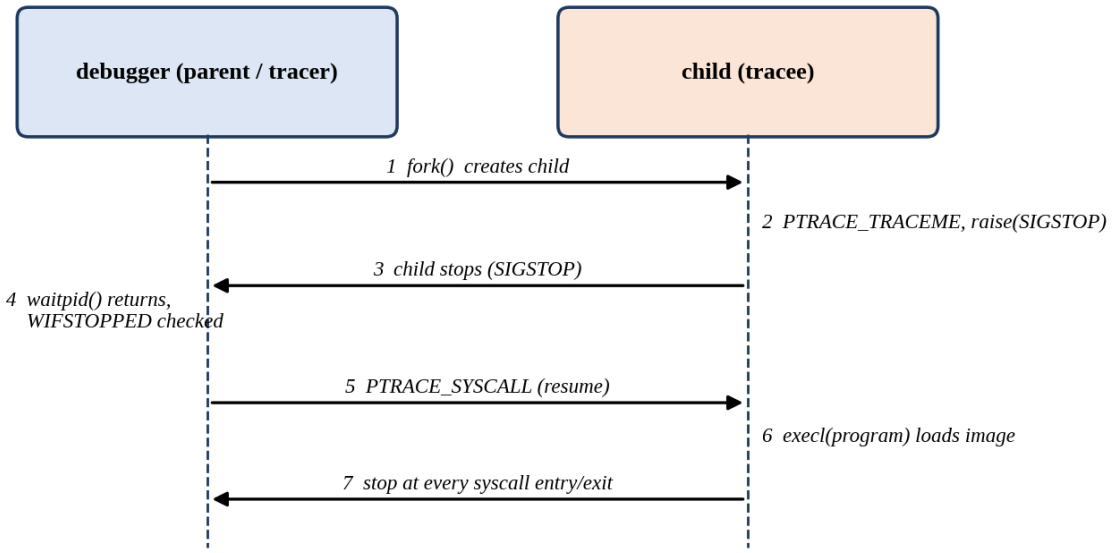


Figure 5.4: Creation and synchronisation of the traced child

5.3.3 Thread monitor data flow

The thread monitor does not use ptrace. The browser sends the PID, the server validates it and lists the directory `/proc/PID/task`, in which each entry is the ID of one thread (Figure 5.5).

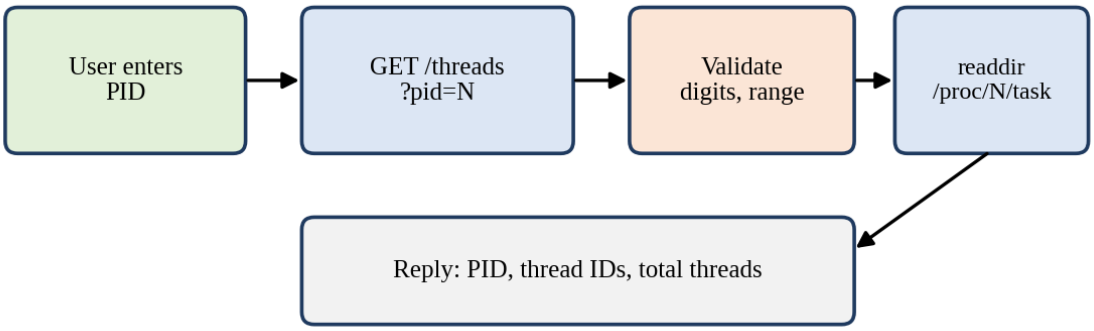


Figure 5.5: Data flow of the POSIX thread monitor

5.4 Module Description

No	Module (file)	Main functions	Kernel interface	Output
1	Main program (debugger.c)	main()	—	Banner and the order of the other modules; “Debugging completed”
2	Process control (process_control.c)	get_target_program, start_target_process, wait_for_initial_stop	fork, ptrace(TRACEME), raise, execl, waitpid	Target name and program, child PID, stop signal number
3	System call tracer (syscall_tracer.c)	trace_system_calls	ptrace(SYSCALL, GETREGSET), waitpid	“[Syscall Entry] Number: N” for every entry; exit information
4	Register reader (register_reader.c)	read_registers	ptrace(GETREGSET, NT_PRSTATUS)	X0–X8, SP, PC and PSTATE in hexadecimal
5	Memory reader (memory_reader.c)	read_process_memory	process_vm_readv	Stack address, bytes read and a hexadecimal dump of 64 bytes
6	Resource monitor (resource_monitor.c)	monitor_resources	getrusage, sysinfo	CPU times, maximum memory, total / free / available RAM
7	Web server (server.c)	run_debugger, save_debug_log, send_debug_history, send_debug_log, send_thread_info, send_process_info, start_thread_demo_if_needed	socket, popen, /proc, opendir	HTTP responses, JSON for history and PIDs, log files
8	Dashboard (web/)	startDebugger, loadDebugHistory, loadThreads, refreshProcessInfo, findThreadDemo, useCurrentPid	fetch (HTTP)	Cards, summary, thread list and history table
9	Sample programs (target/)	target_program main; thread_demo worker threads	pthread_create, pthread_join	The programs that are debugged and observed

5.5 Input and Output Design

5.5.1 Inputs

Input	Provided by	Validation
Target program (dropdown)	User in the dashboard	JavaScript allows only target_program, ls, whoami; the server and debugger repeat the check
TARGET_PROGRAM environment variable	Server, set before popen	Debugger accepts ./target_program, /bin/ls, /usr/bin/whoami; anything else falls back to the default with a message
Process ID (thread monitor)	User in the dashboard	Server accepts digits only and a value between 1 and 4194304
Log file name	Browser (View Log button)	Server accepts only the pattern debug_YYYYMMDD_HHMMSS.log

5.5.2 HTTP interface

Request	Function	Reply
GET /run?program=NAME	Run the debugger and save a log	Text report of the debugger
GET /history	List saved sessions, newest first	JSON list of file name and program
GET /log?file=NAME	Show one saved log	Text of the log (up to 262143 bytes)
GET /process-info	PID and PPID of the server	{"pid":N,"ppid":N}
GET /find-thread-demo	Find or start thread_demo	{"pid":N} or {"started":true,"pid":N}
GET /threads?pid=N	List thread IDs of process N	Text: process ID, thread IDs, total threads
any other GET	Check that the server is alive	“Linux System Debugging Tool Server is running.”

5.5.3 Output of the debugger

The debugger writes a fixed text format. Each module prints a heading framed by lines of “=” characters, which the dashboard recognises: `[Target Process]` lines, `[Syscall Entry] Number: N` lines, the headings `CPU REGISTERS` and `MEMORY READER`, and `[Resource]` lines. The log file stores the same text after the line `Selected program: NAME`.

Structure of the debugger output

```
====================================
     LINUX SYSTEM DEBUGGING TOOL
====================================
[Target Process] Name    : target_program
[Target Process] Program : ./target_program
[Debugger] Child PID: <pid>
[Debugger] Child stopped by signal: <n>
        SYSTEM CALL TRACER
[Syscall Entry] Number: <n>
          CPU REGISTERS          (X0..X8, SP, PC, PSTATE)
          MEMORY READER          (stack address, bytes read, 64 bytes in hex)
  ... repeated for every system call entry ...
[Tracer] Process exited with status: <code>
         RESOURCE MONITOR        ([Resource] CPU User Time, CPU System Time,
                                  Maximum Memory, Total RAM, Free RAM, Available RAM)
[Debugger] Debugging completed.
```

CHAPTER - 6
IMPLEMENTATION

6.1 Module-wise Implementation

6.1.1 Main program (debugger.c)

The main program prints a banner and calls the other modules in order: `start_target_process()`, `wait_for_initial_stop()`, `trace_system_calls()` and `monitor_resources()`. If the child cannot be created or does not stop correctly, the program returns 1 without tracing. Otherwise it ends with the message “Debugging completed”.

6.1.2 Process control (process_control.c)

Input: the environment variable `TARGET_PROGRAM`. **Processing:** `get_target_program()` returns the requested program only if it is exactly `./target_program`, `/bin/ls` or `/usr/bin/whoami`; otherwise it prints a warning and uses `./target_program`. `print_target_process()` prints the name (the part after the last slash) and the program. `start_target_process()` forks; in the child it calls `ptrace(PTRACE_TRACEME)`, raises `SIGSTOP` and calls `execl()`; in the parent it returns the child PID. `wait_for_initial_stop()` prints the PID, waits with `waitpid()` and checks `WIFSTOPPED`. **Output:** the child PID and the stop signal number, or an error.

6.1.3 System call tracer (syscall_tracer.c)

Input: the PID of the stopped child. **Processing:** the function resumes the child with `PTRACE_SYSCALL` and loops on `waitpid()`. If the status shows exit or termination by a signal it prints the code and leaves the loop. If the child is stopped it reads the registers with `PTRACE_GETREGSET`; when the flag `entering` is set it prints the system call number from `regs.regs[8]` and calls `read_registers()` and `read_process_memory()`. The flag is then inverted and the child is resumed. **Output:** one “[Syscall Entry]” block per entry stop.

6.1.4 Register reader (register_reader.c)

Processing: `PTRACE_GETREGSET` with the note type `NT_PRSTATUS` and an `iovec` that points to a `struct user_pt_regs` fills the register structure. The function prints X0 to X8, the stack pointer, the program counter and the processor state as 16-digit hexadecimal values.

6.1.5 Memory reader (memory_reader.c)

Processing: the function first reads the registers to obtain the stack pointer. It then prepares a 64-byte local buffer and two `iovec` structures, one describing the local buffer and one describing the same number of bytes at the target’s stack pointer, and calls `process_vm_readv()`. It prints the stack address, the number of bytes read and the bytes in hexadecimal, sixteen per line. If the call fails, the error is printed with `perror`.

6.1.6 Resource monitor (resource_monitor.c)

Processing: `getrusage(RUSAGE_SELF)` supplies the user and system CPU times and the maximum resident set size; `sysinfo()` supplies total and free RAM, which are converted from bytes to megabytes. The values are printed with the prefix `[Resource]`.

6.1.7 Web server (server.c)

The server creates a TCP socket, sets `SO_REUSEADDR`, binds to port 8080, listens and then loops on `accept()`. For every connection it reads the request into a buffer, decides what to do from the first bytes of the request line and sends a response with the headers `Access-Control-Allow-Origin: *` so that the dashboard page can call it from a local file. Section 6.3 shows the main functions.

6.2 Algorithm / Pseudocode

6.2.1 Creating and controlling the traced child

Algorithm 1: start and synchronise the target

```
ALGORITHM StartTarget
  program <- environment TARGET_PROGRAM
  IF program is not one of {./target_program, /bin/ls, /usr/bin/whoami}
      program <- ./target_program
  child <- fork()
  IF child = 0 THEN                       // inside the child
      ptrace(TRACEME)
      raise(SIGSTOP)                      // wait for the debugger
      execl(program)                      // become the target program
      print error; exit(1)
  RETURN child
  // parent
  waitpid(child)
  IF child stopped THEN print stop signal ELSE error
```

6.2.2 System call tracing

Algorithm 2: trace system calls

```
ALGORITHM TraceSystemCalls(child)
  entering <- 1
  ptrace(SYSCALL, child)
  LOOP
      waitpid(child, status)
      IF child exited OR killed by signal THEN print result; EXIT LOOP
      IF child stopped THEN
          regs <- ptrace(GETREGSET, child)
          IF entering THEN
              print "System call number" = regs.x8
              read_registers(child)
              read_process_memory(child)
          entering <- NOT entering
          ptrace(SYSCALL, child)         // run to the next entry/exit
  END LOOP
```

6.2.3 Reading target memory

Algorithm 3: read the target stack

```
ALGORITHM ReadMemory(child)
  regs <- ptrace(GETREGSET, child)
  local  <- buffer of 64 bytes
  remote <- address regs.sp, length 64
  n <- process_vm_readv(child, local, remote)
  IF n = -1 THEN perror ELSE print address, n and the bytes in hex
```

6.2.4 Running a session on the server

Algorithm 4: run_debugger on the server

```
ALGORITHM RunDebugger(client, name)
  map name to path: ls -> /bin/ls, whoami -> /usr/bin/whoami, else ./target_program
  setenv(TARGET_PROGRAM, path)
  fp <- popen("./debugger", "r")
  unsetenv(TARGET_PROGRAM)
  WHILE fgets(line, fp): append line to result (stop if buffer is full)
  pclose(fp)
  save result as logs/debug_YYYYMMDD_HHMMSS.log with "Selected program: name"
  send result to the client
```

6.3 Core C / C++ Implementation

This section reproduces the source code of the project. The listings are taken directly from the files in the project archive.

6.3.1 debugger.c

```
#include <stdio.h>
#include <sys/types.h>

pid_t start_target_process();
int wait_for_initial_stop(pid_t child);
void trace_system_calls(pid_t child);
void monitor_resources();

int main()
{
    printf("====================================\n");
    printf("     LINUX SYSTEM DEBUGGING TOOL\n");
    printf("====================================\n");

    pid_t child = start_target_process();

    if (child == -1)
    {
        return 1;
    }

    if (wait_for_initial_stop(child) == -1)
    {
        return 1;
    }

    trace_system_calls(child);

    monitor_resources();

    printf("\n[Debugger] Debugging completed.\n");

    return 0;
}
```

6.3.2 process_control.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/ptrace.h>
#include <signal.h>

#define DEFAULT_PROGRAM "./target_program"

static const char *get_target_program(void)
{
    const char *requested = getenv("TARGET_PROGRAM");

    if (requested == NULL || requested[0] == '\0')
    {
        return DEFAULT_PROGRAM;
    }

    /* Only allow these known programs. */
    if (strcmp(requested, "./target_program") == 0 ||
        strcmp(requested, "/bin/ls") == 0 ||
        strcmp(requested, "/usr/bin/whoami") == 0)
    {
        return requested;
    }

    fprintf(stderr,
            "[Process] Unsupported target program; using default.\n");

    return DEFAULT_PROGRAM;
}
```

```
void print_target_process(void)
{
    const char *program = get_target_program();
    const char *name = strrchr(program, '/');

    name = (name != NULL) ? name + 1 : program;

    printf("[Target Process] Name    : %s\n", name);
    printf("[Target Process] Program : %s\n", program);
}

pid_t start_target_process(void)
{
    const char *program = get_target_program();

    print_target_process();

    pid_t child = fork();

    if (child == -1)
    {
        perror("[Process] fork failed");
        return -1;
    }

    if (child == 0)
    {
        if (ptrace(PTRACE_TRACEME, 0, NULL, NULL) == -1)
        {
            perror("[Process] PTRACE_TRACEME failed");
            _exit(1);
        }

        raise(SIGSTOP);

        execl(program, program, (char *)NULL);

        perror("[Process] execl failed");
        _exit(1);
    }

    return child;
}

int wait_for_initial_stop(pid_t child)
{
    int status;

    printf("[Debugger] Child PID: %d\n", child);
    printf("[Debugger] Waiting for child...\n");

    if (waitpid(child, &status, 0) == -1)
    {
        perror("[Debugger] waitpid failed");
        return -1;
    }

    if (WIFSTOPPED(status))
    {
        printf("[Debugger] Child stopped by signal: %d\n",
               WSTOPSIG(status));

        return 0;
    }

    printf("[Debugger] Child did not stop correctly.\n");

    return -1;
}
```

6.3.3 syscall_tracer.c

```
#include <stdio.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/uio.h>
#include <sys/user.h>
#include <asm/ptrace.h>
#include <elf.h>
#include <errno.h>
```

```
void read_registers(pid_t child);
void read_process_memory(pid_t child);

void trace_system_calls(pid_t child)
{
    int status;
    int entering = 1;

    printf("\n====================================\n");
    printf("        SYSTEM CALL TRACER\n");
    printf("====================================\n");

    if (ptrace(PTRACE_SYSCALL, child, NULL, NULL) == -1)
    {
        perror("PTRACE_SYSCALL");
        return;
    }

    while (1)
    {
        if (waitpid(child, &status, 0) == -1)
        {
            perror("waitpid");
            break;
        }

        if (WIFEXITED(status))
        {
            printf("[Tracer] Process exited with status: %d\n",
                   WEXITSTATUS(status));
            break;
        }

        if (WIFSIGNALED(status))
        {
            printf("[Tracer] Process terminated by signal: %d\n",
                   WTERMSIG(status));
            break;
        }

        if (WIFSTOPPED(status))
        {
            struct user_pt_regs regs;
            struct iovec iov;

            iov.iov_base = &regs;
            iov.iov_len = sizeof(regs);

            if (ptrace(PTRACE_GETREGSET,
                       child,
                       (void *)NT_PRSTATUS,
                       &iov) == -1)
            {
                perror("PTRACE_GETREGSET");
                break;
            }

            if (entering)
{
    printf("[Syscall Entry] Number: %lld\n",
           (long long)regs.regs[8]);

    if (entering == 1)
    {
        read_registers(child);
        read_process_memory(child);
    }
}

            entering = !entering;

            if (ptrace(PTRACE_SYSCALL, child, NULL, NULL) == -1)
            {
                perror("PTRACE_SYSCALL");
                break;
            }
        }
    }
}
```

6.3.4 register_reader.c

```
#include <stdio.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/uio.h>
#include <asm/ptrace.h>
#include <elf.h>

void read_registers(pid_t child)
{
    struct user_pt_regs regs;
    struct iovec iov;

    iov.iov_base = &regs;
    iov.iov_len = sizeof(regs);

    if (ptrace(PTRACE_GETREGSET,
               child,
               (void *)NT_PRSTATUS,
               &iov) == -1)
    {
        perror("PTRACE_GETREGSET");
        return;
    }

    printf("\n====================================\n");
    printf("          CPU REGISTERS\n");
    printf("====================================\n");

    printf("X0  : 0x%016llx\n", (unsigned long long)regs.regs[0]);
    printf("X1  : 0x%016llx\n", (unsigned long long)regs.regs[1]);
    printf("X2  : 0x%016llx\n", (unsigned long long)regs.regs[2]);
    printf("X3  : 0x%016llx\n", (unsigned long long)regs.regs[3]);
    printf("X4  : 0x%016llx\n", (unsigned long long)regs.regs[4]);
    printf("X5  : 0x%016llx\n", (unsigned long long)regs.regs[5]);
    printf("X6  : 0x%016llx\n", (unsigned long long)regs.regs[6]);
    printf("X7  : 0x%016llx\n", (unsigned long long)regs.regs[7]);
    printf("X8  : 0x%016llx\n", (unsigned long long)regs.regs[8]);

    printf("SP  : 0x%016llx\n", (unsigned long long)regs.sp);
    printf("PC  : 0x%016llx\n", (unsigned long long)regs.pc);

    printf("PSTATE : 0x%016llx\n",
           (unsigned long long)regs.pstate);
}
```

6.3.5 memory_reader.c

```
#define _GNU_SOURCE

#include <stdio.h>
#include <sys/types.h>
#include <sys/uio.h>
#include <sys/ptrace.h>
#include <asm/ptrace.h>
#include <elf.h>
#include <string.h>
#include <unistd.h>

void read_process_memory(pid_t child)
{
    struct user_pt_regs regs;
    struct iovec reg_iov;

    reg_iov.iov_base = &regs;
    reg_iov.iov_len = sizeof(regs);

    if (ptrace(PTRACE_GETREGSET,
               child,
               (void *)NT_PRSTATUS,
               &reg_iov) == -1)
    {
        perror("[Memory] PTRACE_GETREGSET");
        return;
    }

    unsigned char buffer[64];

    struct iovec local_iov;
    struct iovec remote_iov;
```

```
    memset(buffer, 0, sizeof(buffer));

    local_iov.iov_base = buffer;
    local_iov.iov_len = sizeof(buffer);

    remote_iov.iov_base = (void *)regs.sp;
    remote_iov.iov_len = sizeof(buffer);

    ssize_t bytes_read = process_vm_readv(
        child,
        &local_iov,
        1,
        &remote_iov,
        1,
        0
    );

    printf("\n====================================\n");
    printf("          MEMORY READER\n");
    printf("====================================\n");

    printf("[Memory] Stack Address: 0x%016llx\n",
           (unsigned long long)regs.sp);

    if (bytes_read == -1)
    {
        perror("[Memory] process_vm_readv");
        return;
    }

    printf("[Memory] Bytes read: %zd\n", bytes_read);
    printf("[Memory] Data:\n");

    for (ssize_t i = 0; i < bytes_read; i++)
    {
        printf("%02x ", buffer[i]);

        if ((i + 1) % 16 == 0)
        {
            printf("\n");
        }
    }

    printf("\n");
}
```

6.3.6 resource_monitor.c

```
#include <stdio.h>
#include <sys/resource.h>
#include <sys/sysinfo.h>

void monitor_resources()
{
    struct rusage usage;
    struct sysinfo info;

    printf("\n====================================\n");
    printf("         RESOURCE MONITOR\n");
    printf("====================================\n");

    if (getrusage(RUSAGE_SELF, &usage) == -1)
    {
        perror("[Resource] getrusage");
        return;
    }

    printf("[Resource] CPU User Time   : %ld.%06ld seconds\n",
           usage.ru_utime.tv_sec,
           usage.ru_utime.tv_usec);

    printf("[Resource] CPU System Time : %ld.%06ld seconds\n",
           usage.ru_stime.tv_sec,
           usage.ru_stime.tv_usec);

    printf("[Resource] Maximum Memory  : %ld KB\n",
           usage.ru_maxrss);

    if (sysinfo(&info) == -1)
    {
        perror("[Resource] sysinfo");
```

```
        return;
    }

    printf("[Resource] Total RAM      : %lu MB\n",
           info.totalram / (1024 * 1024));

    printf("[Resource] Free RAM       : %lu MB\n",
           info.freeram / (1024 * 1024));

    printf("[Resource] Available RAM : %lu MB\n",
           (info.freeram * info.mem_unit) / (1024 * 1024));
}
```

6.3.7 target_program.c (default target)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    printf("Target Program Started\n");

    int numbers[5] = {10, 20, 30, 40, 50};

    printf("Processing data...\n");

    for (int i = 0; i < 5; i++)
    {
        printf("Value: %d\n", numbers[i]);

    }

    printf("Target Program Completed\n");

    return 0;
}
```

6.3.8 thread_demo.c (thread demonstration)

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *worker(void *arg)
{
    int id = *(int *)arg;

    printf("[Thread %d] Started\n", id);

    for (int i = 1; i <= 3; i++)
    {
        printf("[Thread %d] Working: step %d\n", id, i);
        sleep(15);
    }

    printf("[Thread %d] Finished\n", id);

    return NULL;
}

int main(void)
{
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};

    printf("=== POSIX THREAD DEMONSTRATION ===\n");
    printf("Main process PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        if (pthread_create(&threads[i], NULL, worker, &ids[i]) != 0)
        {
            fprintf(stderr, "Failed to create thread %d\n", i + 1);
            return 1;
        }
    }

    for (int i = 0; i < 3; i++)
    {
        pthread_join(threads[i], NULL);
```

```
    }

    printf("All three threads completed successfully.\n");

    return 0;
}
```

6.3.9 server.c – send_response and save_debug_log

```
void send_response(int client, const char *body)
{
    char header[512];

    snprintf(header, sizeof(header),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/plain; charset=utf-8\r\n"
        "Access-Control-Allow-Origin: *\r\n"
        "Access-Control-Allow-Methods: GET, OPTIONS\r\n"
        "Access-Control-Allow-Headers: Content-Type\r\n"
        "Content-Length: %zu\r\n"
        "Connection: close\r\n"
        "\r\n",
        strlen(body));

    write(client, header, strlen(header));
    write(client, body, strlen(body));
}

void save_debug_log(const char *result, const char *program)
{
    time_t now = time(NULL);
    struct tm *time_info = localtime(&now);
    char filename[160];

    if (time_info == NULL) {
        fprintf(stderr, "[Log] Failed to get current time.\n");
        return;
    }

    if (strftime(filename, sizeof(filename),
                 "logs/debug_%Y%m%d_%H%M%S.log",
                 time_info) == 0) {
        fprintf(stderr, "[Log] Could not create filename.\n");
        return;
    }

    FILE *log_file = fopen(filename, "w");

    if (log_file == NULL) {
        perror("[Log] Could not create log file");
        return;
    }

    fprintf(log_file, "Selected program: %s\n\n", program);
    fprintf(log_file, "%s", result);
    fclose(log_file);

    printf("[Log] Debugging output saved to: %s\n", filename);
}
```

6.3.10 server.c – run_debugger

```
void run_debugger(int client, const char *program)
{
    FILE *fp;
    char output[4096];
    char result[BUFFER_SIZE] = "";

    const char *program_path;

    if (strcmp(program, "ls") == 0) {
        program_path = "/bin/ls";
    } else if (strcmp(program, "whoami") == 0) {
        program_path = "/usr/bin/whoami";
    } else {
        program = "target_program";
        program_path = "./target_program";
    }

    /*
     * The program name comes only from the allowlist above.
```

```
     * The debugger reads TARGET_PROGRAM when it starts.
     */
    if (setenv("TARGET_PROGRAM", program_path, 1) != 0) {
        send_response(client, "Failed to set target program.");
        return;
    }

    fp = popen("./debugger", "r");

    /*
     * popen has started its child process, which inherits
     * the environment. Remove the variable from the server.
     */
    unsetenv("TARGET_PROGRAM");

    if (fp == NULL) {
        send_response(client, "Failed to start debugger.");
        return;
    }

    while (fgets(output, sizeof(output), fp) != NULL) {
        size_t used = strlen(result);
        size_t incoming = strlen(output);

        if (used + incoming < sizeof(result) - 1) {
            memcpy(result + used, output, incoming + 1);
        } else {
            fprintf(stderr,
                    "[Warning] Debugger output reached buffer limit.\n");
            break;
        }
    }

    int status = pclose(fp);

    size_t len = strlen(result);

    while (len > 0 &&
           (result[len - 1] == '\n' || result[len - 1] == '\r')) {
        result[--len] = '\0';
    }

    if (len + 1 < sizeof(result)) {
        result[len++] = '\n';
        result[len] = '\0';
    }

    save_debug_log(result, program);

    if (status == -1) {
        fprintf(stderr,
                "[Server] Failed to collect debugger exit status.\n");
    } else if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        fprintf(stderr,
                "[Server] Debugger exited with status %d.\n",
                WEXITSTATUS(status));
    }

    send_response(client, result);
}
```

6.3.11 server.c – send_thread_info

```
void send_thread_info(int client, const char *pid_text)
{
    if (pid_text == NULL || *pid_text == '\0') {
        send_response(client, "Please provide a process ID.");
        return;
    }

    for (const char *p = pid_text; *p != '\0'; p++) {
        if (*p < '0' || *p > '9') {
            send_response(client, "Invalid process ID.");
            return;
        }
    }

    long pid = strtol(pid_text, NULL, 10);

    if (pid <= 0 || pid > 4194304) {
        send_response(client, "Invalid process ID.");
```

```
        return;
    }

    char path[128];
    snprintf(path, sizeof(path), "/proc/%ld/task", pid);

    DIR *dir = opendir(path);

    if (dir == NULL) {
        send_response(client,
                      "Process not found or access denied. "
                      "Make sure the process is still running.");
        return;
    }

    char result[8192];
    snprintf(result, sizeof(result),
             "Process ID: %ld\nThread IDs:\n", pid);

    int count = 0;
    struct dirent *entry;

    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] < '0' ||
            entry->d_name[0] > '9') {
            continue;
        }

        char *end;
        long tid = strtol(entry->d_name, &end, 10);

        if (*end != '\0' || tid <= 0) {
            continue;
        }

        char line[64];
        snprintf(line, sizeof(line), "%ld\n", tid);

        if (strlen(result) + strlen(line) < sizeof(result) - 1) {
            strcat(result, line);
            count++;
        }
    }

    closedir(dir);

    char summary[80];
    snprintf(summary, sizeof(summary),
             "Total threads: %d\n", count);

    if (strlen(result) + strlen(summary) < sizeof(result)) {
        strcat(result, summary);
    }

    send_response(client, result);
}
```

6.3.12 server.c – creating and starting the socket server (main, first part)

```
int main(void)
{
    int server_fd;
    int client;

    struct sockaddr_in address;
    char buffer[4096];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        perror("Socket failed");
        return 1;
    }

    int option = 1;

    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,
               &option, sizeof(option));

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
```

```
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address,
             sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 5) < 0) {
        perror("Listen failed");
        close(server_fd);
        return 1;
    }

    printf("====================================\n");
    printf("     LINUX DEBUGGER WEB SERVER\n");
    printf("====================================\n");
    printf("Server running at http://localhost:%d\n", PORT);
    printf("Supported targets: target_program, ls, whoami\n");

    while (1) {
        socklen_t address_length = sizeof(address);

        client = accept(server_fd,
                        (struct sockaddr *)&address,
                        &address_length);

        if (client < 0) {
            perror("Accept failed");
            continue;
        }
```

6.3.13 script.js – sending the run request and extracting values (lines 1–75)

```
async function startDebugger() {
    const output = document.getElementById("debuggerOutput");
    const statusMessage = document.getElementById("statusMessage");
    const processStatus = document.getElementById("processStatus");
    const syscallCount = document.getElementById("syscallCount");
    const registerStatus = document.getElementById("registerStatus");
    const memoryStatus = document.getElementById("memoryStatus");
    const resourceStatus = document.getElementById("resourceStatus");

    // Get selected target program
    const selectedProgram =
        document.getElementById("programSelect").value;

    // Only allow the three supported choices
    const allowedPrograms = ["target_program", "ls", "whoami"];

    if (!allowedPrograms.includes(selectedProgram)) {
        statusMessage.textContent = "Invalid target program.";
        return;
    }

    // Update dashboard
    processStatus.textContent = "Running";
    registerStatus.textContent = "Reading";
    memoryStatus.textContent = "Reading";
    resourceStatus.textContent = "Running";
    output.textContent = "Starting debugger...";
    statusMessage.textContent =
        `Starting debugger for ${selectedProgram}...`;

    try {
        // Send selected program to the C server
        const url =
            `http://localhost:8080/run?program=${encodeURIComponent(selectedProgram)}`;

        const response = await fetch(url);

        if (!response.ok) {
            throw new Error("Server error: " + response.status);
        }

        const text = await response.text();

        // Display target process information
        const targetName =
```

```
        text.match(/\[Target Process\]\s*Name\s*:\s*(.*)/)?.[1]?.trim()
        || "Unknown";

    const targetProgram =
        text.match(/\[Target Process\]\s*Program\s*:\s*(.*)/)?.[1]?.trim()
        || "Unknown";

    document.getElementById("targetName").textContent = targetName;
    document.getElementById("targetProgram").textContent = targetProgram;

    // Count system calls
    const syscallMatches =
        text.match(/\[Syscall Entry\]/g);

    const syscallTotal = syscallMatches
        ? syscallMatches.length
        : 0;

    syscallCount.textContent = syscallTotal;

    // Check debugger modules
    const registersRead = text.includes("CPU REGISTERS");
    const memoryRead = text.includes("MEMORY READER");
    const resourcesRead = text.includes("RESOURCE MONITOR");

    registerStatus.textContent =
        registersRead ? "Read" : "Not Read";

    memoryStatus.textContent =
        memoryRead ? "Read" : "Not Read";
```

6.4 Linux System Calls / APIs Used

Chapter 4 listed the calls. This section explains how the more important ones are used and why.

6.4.1 ptrace()

The signature is `long ptrace(request, pid, addr, data)`. The project uses three requests. `PTRACE_TRACEME` is called by the child: it tells the kernel that the parent is the tracer. `PTRACE_SYSCALL` resumes the stopped tracee and arranges for it to stop at the next system call entry or exit; the last two arguments (`NULL`) mean that no signal is delivered. `PTRACE_GETREGSET` with `addr = NT_PRSTATUS` and `data` pointing to an `iovec` copies the general-purpose registers of the tracee into a `user_pt_regs` structure. On ARM64 the older `PTRACE_GETREGS` request does not exist, which is why `GETREGSET` is used.

6.4.2 waitpid()

`waitpid(child, &status, 0)` blocks until the child changes state. The status word is decoded with macros: `WIFSTOPPED` and `WSTOPSIG` for stops, `WIFEXITED` and `WEXITSTATUS` for a normal exit, `WIFSIGNALED` and `WTERMSIG` for death by a signal.

6.4.3 process_vm_readv()

`process_vm_readv(pid, local_iov, 1, remote_iov, 1, 0)` copies bytes from the address space of process `pid` into a local buffer in one call. It returns the number of bytes copied or −1 on error. The caller needs the same permission that `ptrace` attach would require, which the debugger has because it is the tracer of the child.

6.4.4 getrusage() and sysinfo()

`getrusage(RUSAGE_SELF, &usage)` fills a `struct rusage`. `sysinfo(&info)` fills a `struct sysinfo` whose fields `totalram` and `freeram` are counted in units of `mem_unit` bytes.

6.4.5 popen() and the socket calls

`popen("./debugger", "r")` creates a pipe, forks, and runs the command through the shell with the pipe connected to its standard output, so that the server can read the debugger's output with `fgets()`. `socket()`, `bind()`, `listen()` and `accept()` create the listening socket and return one connected socket per request.

6.5 Process / Thread Implementation

6.5.1 Process model of the tracing session

A tracing session involves up to four processes: the browser, the server, the debugger and the target. The server is the parent of a shell that runs `./debugger` (created by `popen`); the debugger is the parent and tracer of the target. The server's PID and PPID, shown in the dashboard (PID 511, PPID 318 in the recorded run), identify the server process in the process tree.

The life of the child, in the notation of the process state diagram, is: **created** (`fork`) → **stopped** (`SIGSTOP`) → **image replaced** (`execl`) → **running / stopped repeatedly** (`PTRACE_SYSCALL` stops) → **terminated** (exit) → **reaped** (`waitpid` reports the exit status). Because the debugger calls `waitpid` on the child until it reports the exit, the child does not remain a zombie.

6.5.2 Threads in the project

`thread_demo.c` creates the threads with `pthread_create(&threads[i], NULL, worker, &ids[i])`. Each thread runs the function `worker` and receives a pointer to its own element of the `ids` array, so the threads do not share a changing variable. While the program runs, `/proc/PID/task` contains four entries: the main thread (whose ID equals the PID) and the three workers. The server starts `thread_demo` when requested: it forks, calls `setsid()` so that the new process is detached from the server's session, redirects standard input, output and error to `/dev/null`, and executes `./thread_demo`.

6.6 IPC / Synchronization Implementation

6.6.1 Mechanisms

Mechanism	Between	Purpose
Environment variable TARGET_PROGRAM	Server → debugger	Tells the debugger which allow-listed program to trace; the server removes it again with unsetenv after popen
Pipe created by popen()	Debugger → server	Carries the text report to the server
Signals (SIGSTOP, ptrace stops)	Kernel → child; child → parent	Stop the child so that the parent can take control
waitpid()	Child → parent	Synchronise the parent with every stop and with the exit
TCP socket and HTTP	Browser ↔ server	Requests and responses
Files in logs/	Server → server (later request)	Persistent history of sessions
pthread_join()	Main thread → workers	Main thread waits for all three workers

6.6.2 Synchronisation between tracer and tracee

The tracer and the tracee never run at the same time in the tracing loop. The tracee runs only between a `PTRACE_SYSCALL` and the next stop; the tracer examines it only while it is stopped. This strict alternation, enforced by the kernel through `waitpid`, is the synchronisation of the debugger and removes the need for locks.

6.6.3 Absence of data races

The server accepts one connection and answers it completely before it accepts the next, and it uses no threads. The dashboard issues its requests from one page. The debugger traces a single child. Consequently there is no shared variable that two flows of control update concurrently, and no mutex, semaphore or condition variable is required.

6.7 Memory Management Implementation

6.7.1 Buffers and bounds

All buffers are fixed-size arrays on the stack or, for the log display, one block obtained with `malloc`. The sizes and the checks that protect them are:

- `unsigned char buffer[64]` in the memory reader; `process_vm_readv` is asked for exactly `sizeof(buffer)` bytes and the printing loop runs only to the number of bytes returned.
- `char result[BUFFER_SIZE]` (262144 bytes) in the server for the debugger output: each line is copied only if `used + incoming < sizeof(result) - 1`; otherwise a warning is printed and reading stops.
- `char json[65536]` and `files[1000][256]` for the history: at most 1000 log names are collected.
- `char result[8192]` for thread lists: each thread line is added only if it fits.
- The request line is parsed with explicit limits (`i < sizeof(program) - 1`) so that long parameters cannot overflow the destination.
- The log viewer allocates 262144 bytes with `malloc`, reads at most 262143 bytes, terminates the string, and calls `free` after sending.

6.7.2 Address space of the target

The target’s memory is never mapped into the debugger. The kernel copies the requested bytes from the target into the debugger’s buffer during `process_vm_readv`, so the debugger only ever sees a copy and cannot corrupt the target by reading. The address that is read is the stack pointer of the target, i.e. the lowest address of the currently used stack, and the 64 bytes above it are the most recently stored stack contents.

6.7.3 Resource clean-up

Files opened with `fopen` are closed with `fclose`, directories opened with `opendir` are closed with `closedir`, the pipe from `popen` is closed with `pclose` (which also reaps the shell process), and the client socket is closed after every request.

6.8 Optional User Interface Implementation

6.8.1 Dashboard layout

The dashboard (`index.html`, `style.css`, `script.js`) has five areas: (1) the **System Debugger Dashboard** panel with the target selector and the **Start Debugger** button; (2) the **Target Process** panel; (3) five status cards (Process Control, System Calls, CPU Registers, Memory, Resources) and the **System Resource Details** panel with six values; (4) the dark **Debugger Summary** panel; and (5) the lower part with **Process Information**, the **POSIX Thread Monitor** and the **Debugging History** table.

6.8.2 Behaviour

- **Start Debugger** sets the cards to “Running” / “Reading”, calls `/run`, then fills every field from the returned text and writes the summary. If the server cannot be reached, the cards show “Error” and the panel tells the user to start `./server`.
- **Refresh Process IDs** calls `/process-info` and shows the PPID and PID of the server.
- **Find thread_demo PID** calls `/find-thread-demo`; **Use Current Process PID** copies the server PID into the input box; **View Threads** calls `/threads?pid=N` and shows the process ID, the thread IDs and the total.
- **Refresh History** calls `/history`, shows the number of saved sessions and fills a table of log file, target program and a **View Log** button that calls `/log?file=NAME`.

CHAPTER - 7
EXECUTION AND DEMONSTRATION

The screenshots in this chapter are the real output of the project: they were produced by running the server and using the dashboard, as described in Section 7.2. The values quoted in the text are read from these screenshots.

7.1 Compilation in Ubuntu

The project archive contains the sources in four folders: `src/debugger`, `src/server`, `src/target` and `src/web`. All executables are placed in one working folder, because the server starts `./debugger`, `./target_program` and `./thread_demo` by relative path and writes its logs to `logs/`.

Compilation commands (run in the project folder)

```
# 1. the debugger: six source files linked into one executable
gcc -Wall -g -o debugger src/debugger/*.c

# 2. the default target program and the thread demonstration
gcc -Wall -g -o target_program src/target/target_program.c
gcc -Wall -g -o thread_demo    src/target/thread_demo.c -pthread

# 3. the web server
gcc -Wall -g -o server src/server/server.c

# 4. the directory in which the server stores its logs
mkdir -p logs
```

The build produces four executables: `debugger`, `server`, `target_program` and `thread_demo`. The supplied executables are 64-bit ARM ELF files (not stripped), dynamically linked with the C library.

7.2 Execution Commands

Purpose	Command
Start the web server (leave it running)	`./server`
Open the dashboard	`Open src/web/index.html in a web browser; the page talks to http://localhost:8080`
Run the debugger directly on the default target	`./debugger`
Run the debugger directly on ls	`TARGET_PROGRAM=/bin/ls ./debugger`
Run the debugger directly on whoami	`TARGET_PROGRAM=/usr/bin/whoami ./debugger`
Start the thread demonstration by hand	`./thread_demo &`
Check the server from the terminal	`curl http://localhost:8080/process-info`
List the threads of a process from the shell	`ls /proc/PID/task`

Operating sequence used for the screenshots: (1) start `./server`; (2) open the dashboard; (3) select **Default Target Program** and press **Start Debugger**; (4) press **Refresh Process IDs**, then **Use Current Process PID** and **View Threads**; (5) open the Debugging History and press **Refresh History**.

7.3 Sample Inputs and Outputs

7.3.1 Test 1: debugging the default target

Item	Value
Input	Target: Default Target Program (target_program); button: Start Debugger
Command executed by the server	TARGET_PROGRAM=./target_program ./debugger
Target process	Name: target_program, Program: ./target_program
Process status	Completed
System calls detected	33
CPU registers	Read
Memory	Read
Resources	Complete
CPU user time	0.006705 seconds
CPU system time	0.000000 seconds
Maximum memory	1408 KB
Total / Free / Available RAM	7724 MB / 6315 MB / 6315 MB
First ten system call numbers	221, 0, 214, 222, 48, 56, 80, 222, 57, 56

7.3.2 Test 2: process information and thread monitor

Item	Value
Input	Process ID 511 (taken with “Use Current Process PID”)
Parent process ID (PPID)	318
Current process ID (PID)	511
Server reply	Successfully retrieved 1 thread(s) for PID 511.
Output	Process ID: 511; Thread IDs: 511; Total threads: 1

7.3.3 Test 3: debugging history

Item	Value
Input	Refresh History
Saved sessions	64
Newest entries	debug_20261102_035017.log (target_program), debug_20261031_082137.log (ls), debug_20261031_080353.log (ls)
Programs in the list	target_program, ls, whoami and unknown (shown when the first line of a log does not name one of the three allowed targets)
Date range visible	13 October 2026 to 2 November 2026

7.4 Screenshots of Execution

7.4.1 Dashboard and target selection

Figure 7.1 shows the dashboard header, the target selector and the Start Debugger button. After the run the message “Debugging completed for target_program.” appears, and the Target Process panel shows the name and the program path.

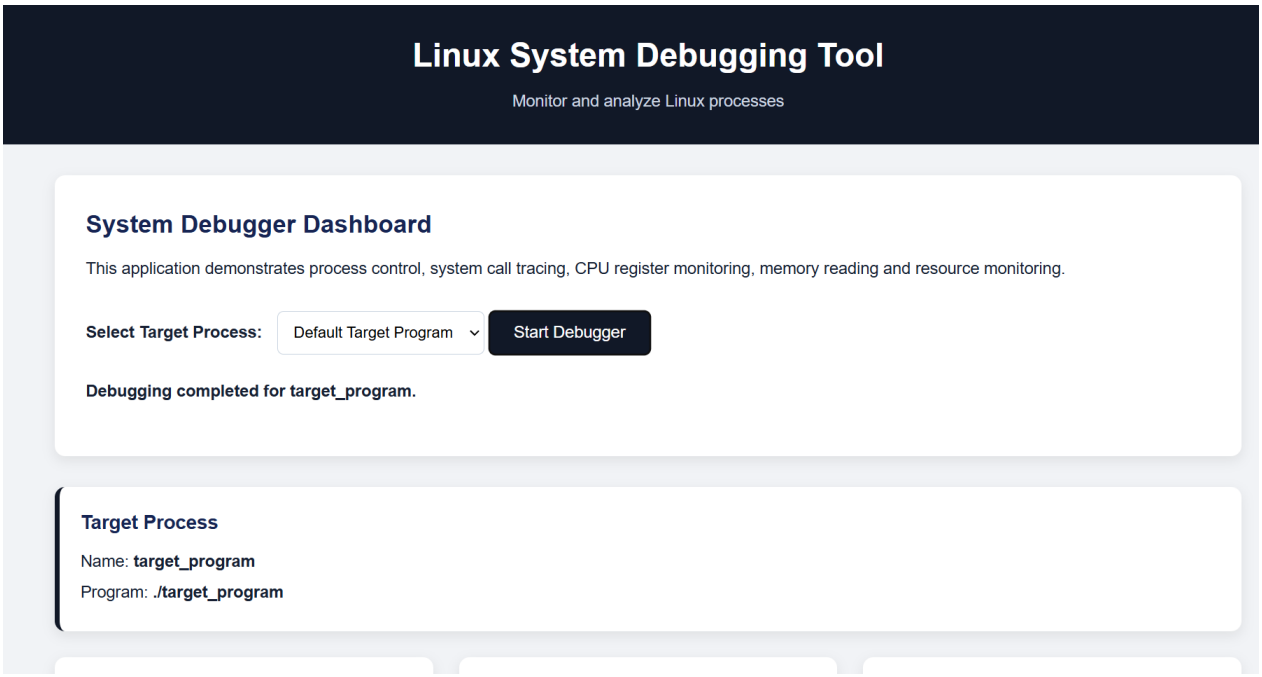


Figure 7.1: System Debugger Dashboard and Target Process panel

7.4.2 Status cards and resource details

Figure 7.2 shows the five status cards (Process Control: Completed, System Calls: 33, CPU Registers: Read, Memory: Read, Resources: Complete) and the six values of the System Resource Details panel, which come from `getrusage()` and `sysinfo()`.

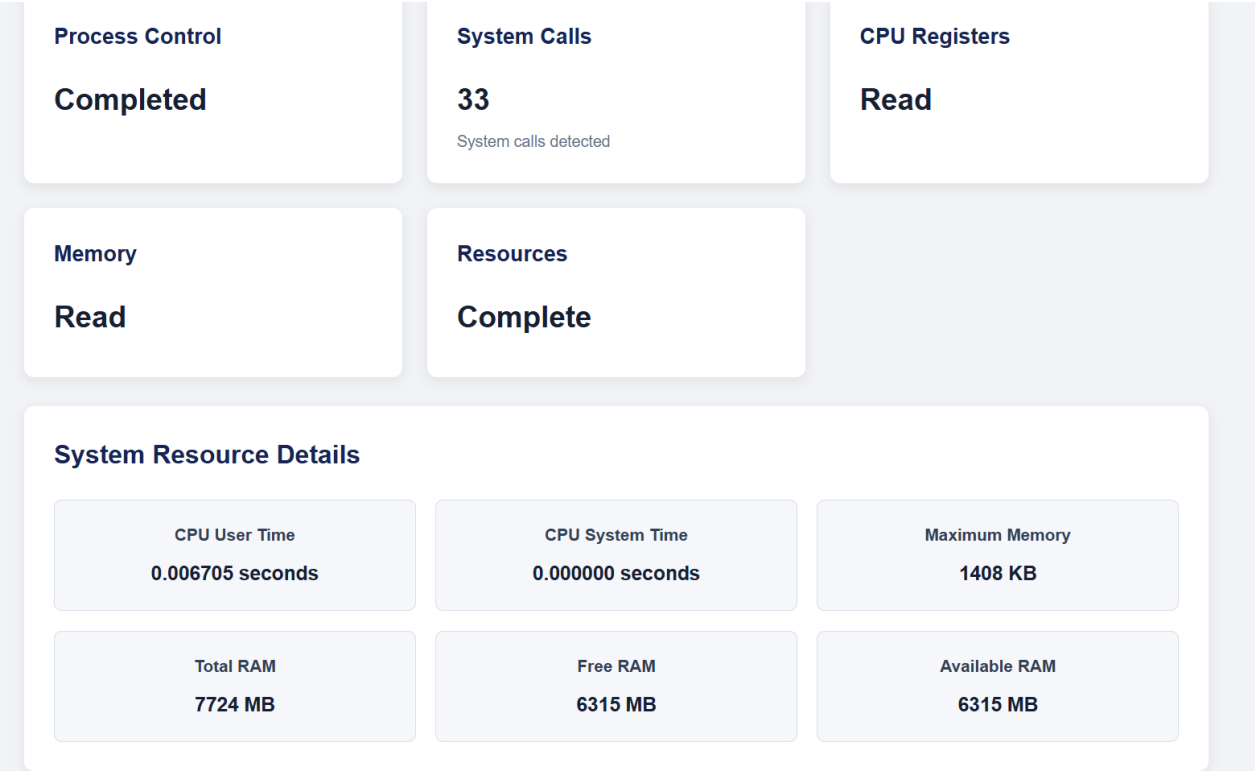


Figure 7.2: Status cards and System Resource Details

7.4.3 Debugger summary

Figure 7.3 shows the Debugger Summary panel. It repeats the target, status and counts, the resource figures and the first ten system call numbers.



Figure 7.3: Debugger Summary (target, resources and first system calls)

7.4.4 Process information and POSIX thread monitor

Figure 7.4 shows the PPID (318) and PID (511) of the server, the input box of the thread monitor and the result for PID 511: one thread, with thread ID 511.

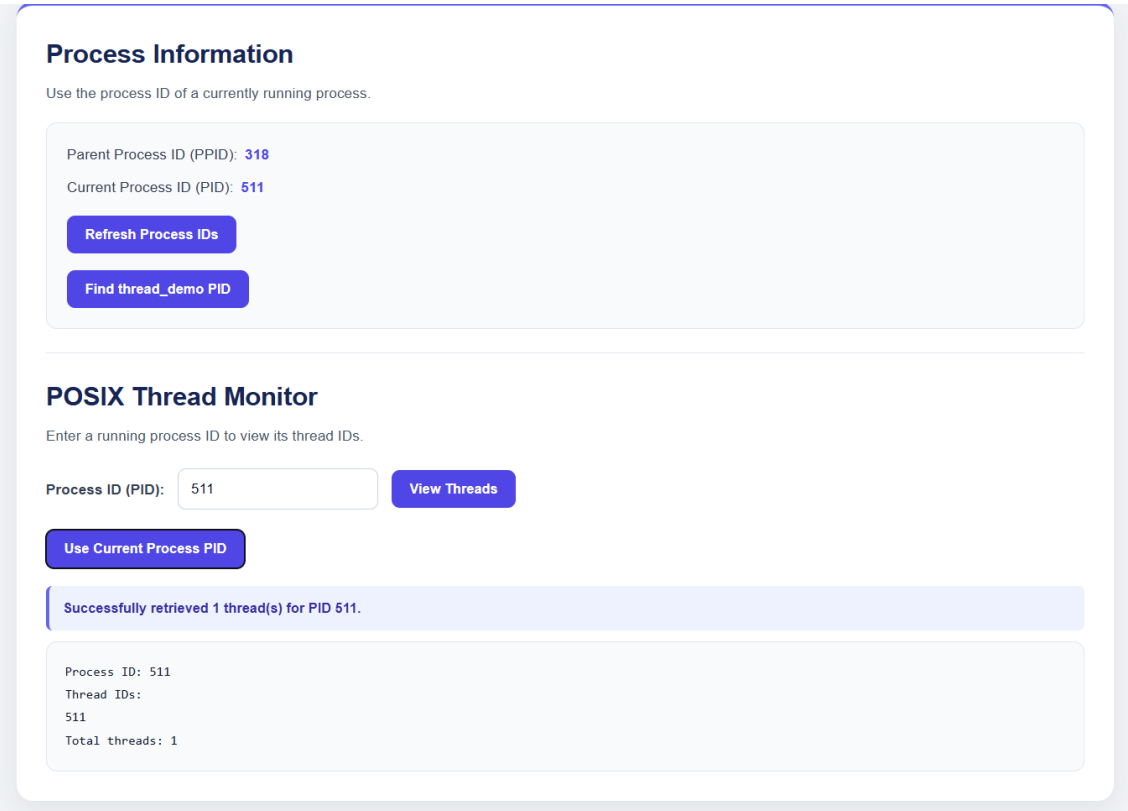


Figure 7.4: Process Information and POSIX Thread Monitor

7.4.5 Debugging history

Figure 7.5 shows the Debugging History table with 64 saved sessions. Each row gives the log file name, the target program and a View Log button.

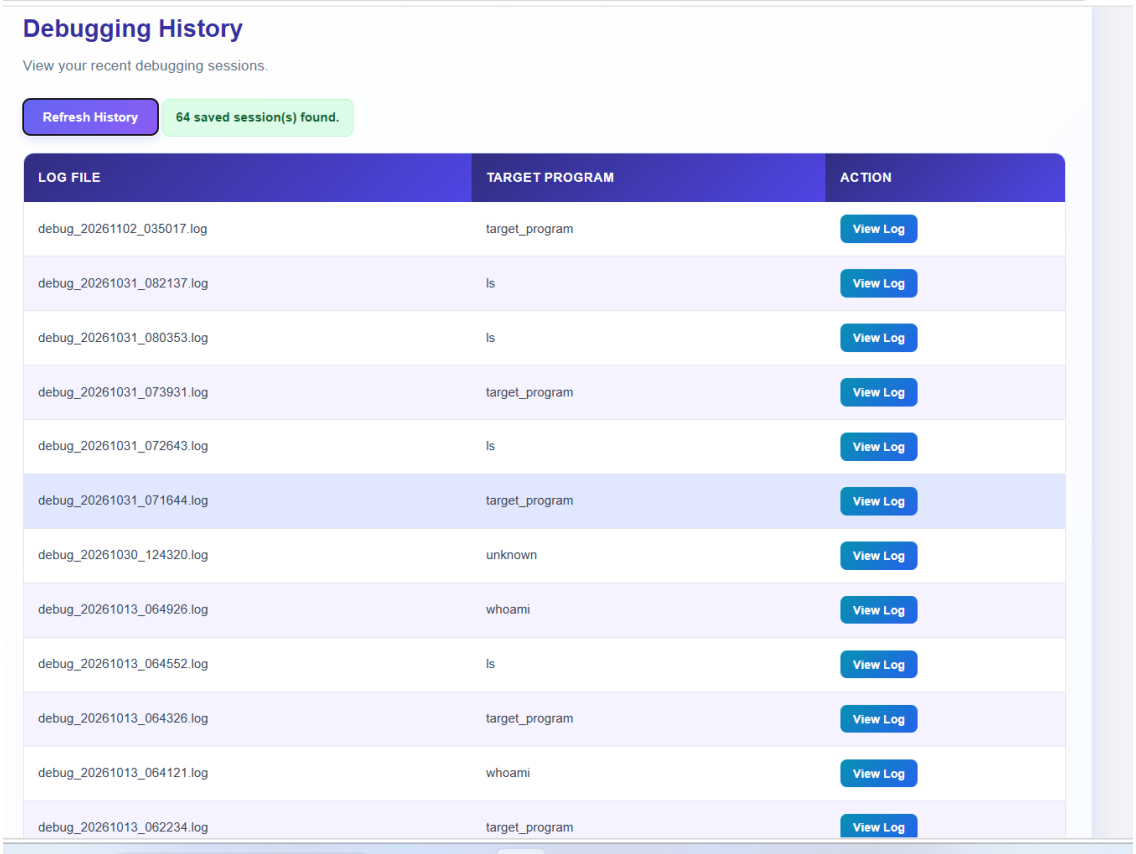


Figure 7.5: Debugging History table with 64 saved sessions

7.5 Demonstration of OS Concepts

OS concept	How the execution demonstrates it	Evidence
Process creation (fork, exec)	The target is started as a child of the debugger and its program image is replaced	Target Process panel: target_program; first system call 221 (execve)
Process life cycle	The child is created, stopped, run, and finally reaped by waitpid	Process Control: Completed
System calls and the kernel boundary	Every system call entry of the child is caught and its number read from register X8	System Calls: 33; numbers 221, 214, 222, 56 ...
Signals	The child stops itself with SIGSTOP; the parent sees the stop through waitpid	Debugger prints “Child stopped by signal” before tracing begins
CPU registers	Register values are read from the stopped child	CPU Registers: Read
Virtual memory	64 bytes of the child’s stack are copied with process_vm_readv	Memory: Read
Resource accounting	CPU time, peak memory and RAM figures come from the kernel	Figure 7.2: 0.006705 s, 1408 KB, 7724 MB
Threads	Thread IDs are listed from /proc/PID/task	Figure 7.4: thread ID 511, total 1
IPC and client-server	Pipe (popen), environment variable and TCP socket carry data	The dashboard receives the debugger text; logs are stored
Persistent storage	Each run is saved as a file and listed later	Figure 7.5: 64 saved sessions

CHAPTER - 8
CONCLUSION

8.1 Summary of the Project

The Linux System Debugging Tool is a compact debugger and monitoring tool that shows how Linux lets one process observe and control another. The debugger creates a child with `fork()`, places it under tracing with `ptrace(PTRACE_TRACEME)`, synchronises through `SIGSTOP` and `waitpid()`, resumes it with `PTRACE_SYSCALL` and reports every system call entry together with the CPU registers and 64 bytes of stack memory read with `process_vm_readv()`. Finally it reports CPU time and memory with `getrusage()` and `sysinfo()`.

A C web server and a browser dashboard make the tool easy to use. The server runs the debugger on an allow-listed target, stores every session in a time-stamped log, lists the history, reports the server's own PID and PPID and lists the threads of a process from `/proc`. The recorded execution traced `target_program`, detected 33 system call entries and showed 64 saved sessions in the history.

8.2 Objectives Achieved

Objective (Section 1.4)	Status	Evidence
Study and apply ptrace, fork, execl, waitpid and signals	Achieved	process_control.c, syscall_tracer.c
Intercept every system call with PTRACE_SYSCALL	Achieved	33 entries counted for target_program
Read CPU registers with PTRACE_GETREGSET	Achieved	register_reader.c; CPU Registers: Read
Read process memory with process_vm_readv	Achieved	memory_reader.c; Memory: Read
Measure CPU time and memory	Achieved	Figure 7.2 (see Section 8.5 for the scope of the measurement)
Modular code base	Achieved	Six debugger source files plus server and web files
Client-server layer with allow-list	Achieved	server.c run_debugger; three allowed targets
List threads of a process	Achieved	Figure 7.4; thread_demo.c
Store sessions and show history	Achieved	Figure 7.5: 64 sessions
Document design and results	Achieved	This report

8.3 Results Achieved

Result	Value
Target debugged	target_program (also ls and whoami selectable)
System call entries detected	33
CPU user time / system time	0.006705 s / 0.000000 s
Maximum memory	1408 KB
Total / free RAM	7724 MB / 6315 MB
Threads of the server process (PID 511)	1
Saved debugging sessions	64 (13 October to 2 November 2026)
Size of the debugger	394 lines of C in six files
Size of the server / dashboard	739 lines of C / 1375 lines of HTML, CSS and JavaScript

8.4 Skills and Learning Outcomes

- Using `ptrace`, `waitpid` and signals to control another process.
- Understanding how system call numbers are passed in registers on ARM64 and how a tracer sees them.
- Reading another process's memory with `process_vm_readv()` and interpreting a hexadecimal dump.
- Using `/proc` to discover processes and threads.
- Designing a client-server application with sockets, `popen` and an HTTP interface in C.
- Writing safe C: bounded buffers, input validation, allow-lists and resource clean-up.
- Dividing a project into modules and presenting its results through a web interface.

8.5 Limitations and Future Scope

8.5.1 Limitations observed in the implementation

- **Resource figures describe the debugger, not the target.** `resource_monitor.c` calls `getrusage(RUSAGE_SELF)`, so the CPU times and maximum memory are those of the debugger process. Using `RUSAGE_CHILDREN` after the child has been reaped, or reading `/proc/PID/stat` of the target, would measure the target.
- **Free RAM and Available RAM are equal.** Both values are computed from the `freeram` field (6315 MB in the recorded run). A true "available" figure, which includes reclaimable cache, needs `MemAvailable` from `/proc/meminfo`.
- **Registers and stack are dumped at every entry.** Because `read_registers()` and `read_process_memory()` are called at each system call entry, the output grows quickly; the 33 entries therefore produce 33 register blocks and 33 memory dumps.
- **A system call number of 0 appears.** The second number in the recorded list is 0, which is not a call a normal program makes. A likely cause is that, after `execve`, the kernel sends an extra `SIGTRAP` stop that shifts the entry/exit alternation of the loop; enabling `PTRACE_O_TRACESYSGOOD` and checking `SIGTRAP | 0x80` would separate real system call stops from other stops. This explanation has not been verified by a test.
- **Only the system call number is shown.** Names, arguments and return values are not decoded.
- **Architecture and platform.** The register code is written for the ARM64 register layout and the server is meant for local use without authentication.
- **Fixed target list.** Only `target_program`, `ls` and `whoami` can be debugged from the dashboard.

8.5.2 Future scope

- Translate system call numbers into names and print arguments and return values.
- Add breakpoints and single-stepping with `PTRACE_SINGLESTEP`.
- Measure the target's own CPU time and memory and show them over time.
- Add an x86-64 version of the register module and select the architecture at compile time.

- Show the memory map of the target by reading `/proc/PID/maps` and allow the user to choose the address to dump.
- Add search and filters to the history table and a comparison of two sessions.
- Add authentication and a larger, configurable list of safe targets.

CHAPTER - 9
REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th Edition, Wiley.
2. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th Edition, Pearson.
3. M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.
4. W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd Edition, Addison-Wesley.
5. R. Love, *Linux Kernel Development*, 3rd Edition, Addison-Wesley.
6. R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd Edition, Pearson.
7. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall.

9.2 Research Papers

1. D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System", *Communications of the ACM*, vol. 17, no. 7, 1974.
2. N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation", *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007.

9.3 Linux / GNU Documentation

1. Linux manual pages: `ptrace(2)`, `fork(2)`, `execve(2)`, `wait(2)` / `waitpid(2)`, `process_vm_readv(2)`, `getrusage(2)`, `sysinfo(2)`, `signal(7)`, `proc(5)`, `pthreads(7)`, `popen(3)`, `socket(2)`.
2. GNU C Library Reference Manual, Free Software Foundation.
3. GCC, the GNU Compiler Collection: user manual and option summary, Free Software Foundation.
4. Linux kernel source: `include/uapi/asm-generic/unistd.h` (system call numbers used on ARM64) and the ptrace documentation in the kernel documentation tree.

9.4 Online Resources

1. Linux man-pages project: https://man7.org/linux/man-pages/
2. The Linux Kernel documentation: https://www.kernel.org/doc/html/latest/
3. GNU Compiler Collection: https://gcc.gnu.org/
4. Project repository: https://github.com/Dharmika-12/KLH-CSE-2026-27-Team15-LinuxSystemDebuggingTool